Product Requirements Document: Boardroom

Product	Boardroom, the Executable Whiteboard
Status	Working prototype, deployed at https://board-of-c.vercel.app
Owner	Boardroom team

1. Background and problem

Programming is taught on whiteboards, but whiteboard code is never run. Students cannot tell whether an example works; teacher slips (missing `&` in `scanf`, off-by-one errors, dangling pointers) go unnoticed; memory concepts such as `malloc()` remain abstract; and moving code into an IDE interrupts the lecture.

2. Goals and non-goals

Goals

1. Let a teacher run what is on the board without leaving the board.
2. Show real compiler, runtime and memory-safety feedback linked to the code.
3. Use a pretrained vision-language model (Qwen2-VL, run locally) for handwriting only, with the teacher confirming every transcription.
4. Preserve every lesson as a reproducible, versioned record.

Non-goals

- Physical-whiteboard camera capture (dropped after early evaluation).
- User accounts, grading, plagiarism detection, or a student-facing exam mode.
- Safely running arbitrary public code at scale.

3. Users

Persona	Need
Teacher (primary)	Demonstrate and debug code live, keep lesson records
Student (secondary)	Review what was run, replay a lesson, see why an error happened
Administrator (future)	Deploy for a department with controlled access

4. User stories and acceptance criteria

ID	Story	Acceptance criteria	Status
U1	As a teacher, I write code on a canvas and run it	Selecting a frame extracts source; Run returns real stdout/stderr	Done
U2	I supply input by writing in blue	Blue ink becomes stdin; other colours are ignored	Done
U3	I write by hand and the app reads it	The local Qwen2-VL model returns editable lines; Run is blocked until both code and input are confirmed	Done

ID	Story	Acceptance criteria	Status
U4	I see errors where they happened	Compiler/ASan/UBSan diagnostics map to source lines; clicking focuses the board when a location exists	Done
U5	I choose C++17 or Python	Language toggle applies to execution, OCR prompts and snapshots	Done
U6	I run several inputs	Up to 20 cases with expected status/output, progress and first-mismatch report	Done
U7	I use teaching data structures	`stack.h`, `linked_list.h`, `queue.h`, `search.h` compile and report contract failures at the teacher's line	Done
U8	I learn what `malloc`/`free` did	Rule-based teaching notes appear for matching calls; comments and strings are ignored	Done
U9	I keep a record of every lesson	Each run creates a Git commit with board, source, input and replay bundle; History lists the latest 30	Done
U10	I replay an old lesson	Frozen runner, vendored headers and SHA-256 manifest rebuild the run offline	Done
U11	I write only loose lines and the app adds the boilerplate	The model adds includes, main and return only; the app verifies each original line is kept in order and unchanged; the teacher confirms the prepared program before it runs	Done

5. Functional requirements

Area	Requirement
Canvas	Excalidraw whiteboard, frames, fullscreen, resizable side panel
Extraction	Typed text read literally with blank lines and a line-to-element map; handwriting exported as separate code and stdin PNGs (max 2048 x 2048)
OCR	Structured lines with uncertainty flags from a pretrained local model (Qwen2-VL); must not repair mistakes; teacher confirmation mandatory
Snippet mode	C++ only. Qwen2-VL adds boilerplate around loose lines and never edits them; an automatic line-by-line check plus teacher confirmation precede execution
Execution	g++ C++17 with `-fsanitize=address,undefined`, or Python 3, inside an isolated container
Limits	32 KiB source, 8 KiB stdin, 10 s compile, 5 s run, 16 KiB output
Archive	Git commit per run, local or GitHub-backed, non-force pushes, retry sync
Hosting	Local Docker, Docker API behind HTTPS, or Vercel Functions with Wandbox

6. Non-functional requirements

Category	Requirement
Security	No network in the sandbox; read-only filesystem; dropped capabilities; non-root user; memory/CPU/process caps; secrets server-side only (see README section 7)
Privacy	No accounts or database; boards leave the browser only on OCR, run or snapshot; handwriting never sent to a third-party API; no AI API key exists in the project; logs exclude images and transcriptions
Performance	Typed code returns a result within the 22 s outer limit; OCR bounded at 50 s

Category	Requirement
Reliability	Outer timeout force-removes containers; snapshot Git failures keep files and warn
Usability	Single-screen workflow; typed input as full alternative to handwriting; keyboard-resizable panel
Maintainability	Minimal dependencies (React, Excalidraw, Vite); Node stdlib API; unit, container and browser tests

7. Technical approach

See the architecture diagram and deployment modes in README section 5 and the detailed technical overview.

8. Risks and mitigations

Risk	Mitigation
OCR misreads handwriting	Mandatory review, uncertainty flags, scene fingerprint invalidation, typed fallback
Model "fixes" teacher errors	Prompt forbids repairs; compilation uses the confirmed text only
Malicious code	Container isolation and resource limits; public use needs added authentication and stronger isolation
Pretrained model misreads handwriting	Mandatory review, uncertainty flags, typed fallback; benchmark on classroom samples next
Hosted runner is third-party	Documented as weaker than Docker; Docker mode recommended for classrooms
Public API abuse	Origin checks, body and rate limits; per-user auth listed as future work

9. Success metrics

- Handwriting to a real result in under one minute for a short program.
- 100% of runs preceded by teacher confirmation when the source is handwritten.
- Compiler errors point to the correct board line for typed code.
- A saved lesson replays from Git alone.

10. Roadmap

Horizon	Items
Next month	Fine-tune the OCR model on code handwriting; per-teacher accounts and rate limits shared across servers (per-visitor limits already exist); stronger sandbox (gVisor/Firecracker); handwritten diagnostics with line boxes; plain C mode
Later	Student view and shared live sessions; more languages (Java); library versioning; teacher analytics on common errors